Lab 11: Government Data APIs

INFO 4617/5617 · Week 11 · Chapter 11, Government Data APIs

Today we code together. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll answer the chapter's question with two public APIs: **how did Boulder's housing come to look the way it does?** The Census Bureau tells us what the housing stock is today; FRED, the Federal Reserve Bank of St. Louis's data service, tells us about the economy it was built in. Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Request free keys from the Census Bureau and FRED (and, for the optional Step 9, the FEC), and set them as the environment variables `CENSUS_API_KEY` and `FRED_API_KEY` before you start Jupyter; Missing Manual Chapter 34 shows how. Approval can take a few minutes, so don't wait until class.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Run live on 2026-10-21. The Census Bureau's cells (Steps 0–4, their microsprints, and Step 8's first cell) ran; their figures come from the 2018–2022 American Community Survey and won't change on a rerun. FRED's cells (Step 5; Step 6's plot and microsprint; Step 7's three cells and its microsprint; Step 8's plot and microsprint) are filled in but tagged `not-run-live`: on 2026-10-21, through at least 22:26 UTC, FRED's API answered every request with a "Site Maintenance" web page and status 200, so `.json()` failed. (Step 6's `fred_series()` definition ran: it sends nothing until it's called.) So nothing in those cells, their series IDs included, has been checked against FRED, and Step 10's first two answers are only half written. Run the key again with `labs/tools/run_key.py` once FRED is back, and finish them from its outputs; FRED requests end on 2026-09-30, so they will match later runs unless FRED revises a past value. What Step 7 says about the chapter's ID comes from the Bureau of Labor Statistics' area list, not from FRED. Step 9 is filled in but wasn't run (tagged `skip-execution`): there was no FEC key. The lab follows the textbook at commit `d9306b7`; where it departs from the chapter, the step says so.

Step 0 — Set up, and keep the keys out of the notebook (5 minutes)

An API key identifies you, counts your requests, and is yours alone. A key typed into a notebook travels with the notebook: into a GitHub repository, a class submission, a classmate's laptop. So read the keys from environment variables, as the chapter's "API Key Management" section does. The chapter checks each key and raises its own error; `os.environ["NAME"]` does the same in one line, since a missing key raises a `KeyError` that names it. (The chapter also shows a second way, a `.gitignore`d `api_keys.json` file; either ends with the same two variables.) Define the User-Agent too: these APIs ask for a key, but every request still says who sent it, in the book's form.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt)
# Define HEADERS, whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)", with your email
# Read CENSUS_API_KEY and FRED_API_KEY into census_key and fred_key with os.environ[...]; never print a key
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
census_key = os.environ["CENSUS_API_KEY"]
```

```
fred_key = os.environ["FRED_API_KEY"]
print("Both keys found")
```

```
Both keys found
```

💬 Discuss

- In week 10 we told Wikipedia who we are with a User-Agent. The Census Bureau and FRED also ask for a key: what can a key do that a User-Agent can't?
- Where could a key typed into this notebook end up: a commit, a screenshot, a shared notebook? Which of those could you take back?
- If `FRED_API_KEY` isn't set, what does this cell do? Why is that better than finding out at the first request?

Step 1 — The variable catalog (7 minutes)

The American Community Survey's thousands of variables come in groups (the chapter's "Navigating the Variable Catalog"). Group `B25034` is "Year Structure Built", and the group's catalog is itself an API: send it a GET with `requests.get()` and our `HEADERS`, and read the answer with `.json()`. Print only the estimates, the codes ending in `E`: the chapter skips the margins of error (`M`), but the group also holds annotation variables (`EA`, `MA`) that would triple the list. Browse all the groups at api.census.gov/data/2022/acs/acs5/groups.html.

```
# Request "https://api.census.gov/data/2022/acs/acs5/groups/B25034.json" with HEADERS; read the
↪  JSON into variables
# Loop over sorted(variables["variables"].items()), and print each code that ends in "E" with its
↪  label
variables_url = "https://api.census.gov/data/2022/acs/acs5/groups/B25034.json"
variables = requests.get(variables_url, headers=HEADERS).json()

for var_id, var_info in sorted(variables["variables"].items()):
    if var_id.endswith("E"):
        print(f"{var_id}: {var_info['label']}")
```

```
B25034_001E: Estimate!!Total:
B25034_002E: Estimate!!Total:!!Built 2020 or later
B25034_003E: Estimate!!Total:!!Built 2010 to 2019
B25034_004E: Estimate!!Total:!!Built 2000 to 2009
B25034_005E: Estimate!!Total:!!Built 1990 to 1999
B25034_006E: Estimate!!Total:!!Built 1980 to 1989
B25034_007E: Estimate!!Total:!!Built 1970 to 1979
B25034_008E: Estimate!!Total:!!Built 1960 to 1969
B25034_009E: Estimate!!Total:!!Built 1950 to 1959
B25034_010E: Estimate!!Total:!!Built 1940 to 1949
B25034_011E: Estimate!!Total:!!Built 1939 or earlier
NAME: Geographic Area Name
```

💬 Discuss

- What do the parts of `B25034_002E` tell you: the table, the line, and the final letter? Why would an estimate need an annotation variable beside it?
- In week 6, The Numbers hid its table among decoys. How is picking a variable from this catalog like that, and how is it different?
- Every cell today assumes its request works. If the Census Bureau answered a 429, or a web page with status 200, which line would fail, and would week 10's `raise_for_status()` have caught either?

Step 2 — Ask about one place (5 minutes)

Places are named by FIPS codes: Colorado is state `08`, and Boulder is place `07850`. The chapter's "Querying for Specific Places" asks for the place's name and the first few `B25034` estimates, with `get`, `for`, and `in` parameters, and the key. The answer is a list of lists whose first row is the header, and every value is a string, so make a DataFrame and turn the numbers into numbers with `pd.to_numeric`. Keep the chapter's names, `census_url` and `numeric_cols`: we use them again.

```
# census_url = "https://api.census.gov/data/2022/acs/acs5"
# params: get="NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
 ↪  for="place:07850", in="state:08", key=census_key
# Request census_url with params and HEADERS; make df from rows 1 onward, with row 0 as the
 ↪  columns
# numeric_cols: the columns that start with "B25034"; convert them with pd.to_numeric, and show
 ↪  df
census_url = "https://api.census.gov/data/2022/acs/acs5"
params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
          "for": "place:07850", "in": "state:08", "key": census_key}
data = requests.get(census_url, params=params, headers=HEADERS).json()

df = pd.DataFrame(data[1:], columns=data[0])
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric)
df
```

```
                    NAME  B25034_001E  B25034_002E  B25034_003E  B25034_004E  \
0  Boulder city, Colorado        46551           68         3784         3847

   B25034_005E state  place
0         5931    08  07850
```

💬 Discuss

- In week 5, `urlparse()` took apart a Census URL just like this request's. Which parameters name the variables, which the place, and which must never appear in a printed error?
- In week 7, the Wayback Machine's CDX API sent the same shape: a list of lists with a header row. Why might an API send data this way rather than as nested dictionaries?
- Why do the numbers arrive as strings, and why must a FIPS code like `08` stay one?

Step 3 — Compare two places (12 minutes)

Several places can go in one request: `place:07850,45970` is Boulder and Longmont (the chapter's "Comparing Multiple Locations"). Change one parameter and ask again, then plot the two cities' housing by decade built, side by side. The chapter's chart includes the total among the decades, which dwarfs them; leave it out, and label the decades.

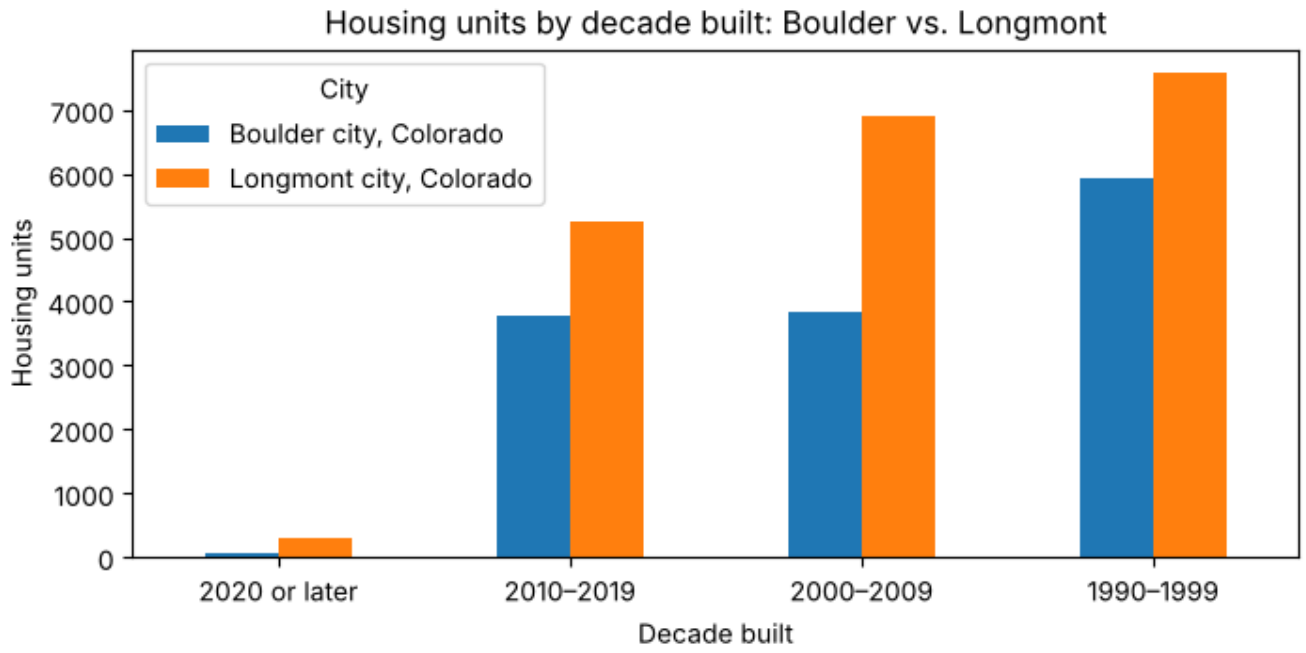
```
# Set params["for"] to "place:07850,45970" and request again
# Build df_compare as in Step 2, convert numeric_cols, and show it
params["for"] = "place:07850,45970"
data = requests.get(census_url, params=params, headers=HEADERS).json()
df_compare = pd.DataFrame(data[1:], columns=data[0])
df_compare[numeric_cols] = df_compare[numeric_cols].apply(pd.to_numeric)
df_compare
```

```
                     NAME  B25034_001E  B25034_002E  B25034_003E  \
0   Boulder city, Colorado        46551           68         3784
1  Longmont city, Colorado        41429          303         5250
```

```
   B25034_004E  B25034_005E state  place
0         3847         5931    08  07850
1         6912         7569    08  45970
```

```
# df_plot: df_compare indexed by NAME, the decade columns only (numeric_cols[1:]), transposed;
 ↪  name its rows by decade
# On plt.subplots(), plot df_plot as grouped bars, with a title, axis labels, and a legend titled
 ↪  "City"
df_plot = df_compare.set_index("NAME")[numeric_cols[1:]].T
df_plot.index = ["2020 or later", "2010–2019", "2000–2009", "1990–1999"]

fig, ax = plt.subplots(figsize=(8, 3.5))
df_plot.plot(kind="bar", ax=ax, rot=0)
ax.set(title="Housing units by decade built: Boulder vs. Longmont",
       xlabel="Decade built", ylabel="Housing units")
ax.legend(title="City")
plt.show()
```



⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 1. Cities differ in size, so raw counts don't compare fairly; shares do.

Do: ask for the total (`B25034_001E`) and the three decades since 2000 (`_002E` to `_004E`) for five Colorado places, and compute the share of each city's housing built in 2000 or later. The key uses Boulder, Longmont, Fort Collins (`27425`), Lakewood (`43000`), and Castle Rock (`12415`); look up others in the FIPS code lists. **Share:** the city with the largest share, the one with the smallest, and the two shares.

```
# Microsprint: set params["get"] to NAME and B25034_001E to B25034_004E, params["for"] to five
 ↪  places, and request
# df_five: indexed by NAME, the four counts converted with pd.to_numeric
# since_2000: 002E + 003E + 004E; share: since_2000 / 001E; show share, largest first
params["get"] = "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E"
params["for"] = "place:07850,45970,27425,43000,12415"
data = requests.get(census_url, params=params, headers=HEADERS).json()
```

```
df_five = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
df_five = df_five[["B25034_001E", "B25034_002E", "B25034_003E",
 ↪  "B25034_004E"]].apply(pd.to_numeric)
since_2000 = df_five["B25034_002E"] + df_five["B25034_003E"] + df_five["B25034_004E"]
share = since_2000 / df_five["B25034_001E"]
share.sort_values(ascending=False).round(3)
```

```
NAME
Castle Rock town, Colorado     0.685
Fort Collins city, Colorado    0.341
Longmont city, Colorado        0.301
Boulder city, Colorado         0.165
Lakewood city, Colorado        0.150
dtype: float64
```

💬 Discuss

- Which of your five cities built the most of its housing since 2000, and which the least? What might explain the difference?
- Longmont out-built Boulder in every decade the chart shows. Is that a fair comparison, and what does the share add?
- These are 2018–2022 five-year estimates. What does that mean for a count labeled ”2020 or later”?

Step 4 — Sanity-check a number (8 minutes)

A variable with the right-sounding label can still be the wrong one. The chapter's ”Finding the Right Variable” ends with a practical tip: before you trust a pipeline, retrieve one value you can judge. Ask for Boulder's median household income, `B19013_001E`. Does it look like a city's median income: tens of thousands of dollars, not 7 or 70 million?

```
# params: get="NAME,B19013_001E", for="place:07850", in="state:08", key=census_key; request it
# Print the city and its median household income, with a dollar sign and commas
params = {"get": "NAME,B19013_001E", "for": "place:07850", "in": "state:08", "key": census_key}
data = requests.get(census_url, params=params, headers=HEADERS).json()
income = int(data[1][1])
print(f"{data[1][0]}: median household income ${income:,}")
```

```
Boulder city, Colorado: median household income $80,243
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Check the step's result another way. Every estimate has a margin of error, the variable with the same code ending in `M`; the American Community Survey gives it at 90% confidence.

Do: ask for `B19013_001E` and `B19013_001M` for Boulder and Longmont, and compute each city's interval: the estimate minus and plus its margin. **Share:** the two intervals, and whether they overlap.

```
# Microsprint: set params["get"] to NAME, B19013_001E, and B19013_001M, params["for"] to
 ↪  "place:07850,45970", and request
# incomes: indexed by NAME, the two columns converted; add low (estimate - margin) and high
 ↪  (estimate + margin)
# Show incomes: do the two intervals overlap?
params["get"] = "NAME,B19013_001E,B19013_001M"
params["for"] = "place:07850,45970"
data = requests.get(census_url, params=params, headers=HEADERS).json()
incomes = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
incomes = incomes[["B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
```

```
incomes["low"] = incomes["B19013_001E"] - incomes["B19013_001M"]
incomes["high"] = incomes["B19013_001E"] + incomes["B19013_001M"]
incomes
```

```
                        B19013_001E  B19013_001M    low   high
NAME
Boulder city, Colorado         80243         3561  76682  83804
Longmont city, Colorado        89720         3791  85929  93511
```

💬 **Discuss**

- Does Boulder's median income look right? What did you compare it with?
- Did the two intervals overlap? If they had, what could you no longer say about which city's households earn more?
- Which checks like this one could a pipeline run on its own, every time it runs?

Step 5 — A FRED series (7 minutes)

FRED holds hundreds of thousands of time series, each with an ID you find by searching its website. The chapter's "Finding Series IDs" finds `MHICO08013A052NCEN`, median household income for Boulder County from the Census Bureau's SAIPE program: the ID embeds the state (`CO`) and the county's FIPS code (`08013`). Ask the `series/observations` endpoint for it, as the chapter does, and fix `observation_end` at a past date so a rerun gives the same answer. FRED writes a missing value as `"."`, so convert with `errors="coerce"`.

```
# fred_url = "https://api.stlouisfed.org/fred/series/observations"
# params: series_id="MHICO08013A052NCEN", api_key=fred_key, file_type="json",
 ↪  observation_end="2026-09-30"; request it
# df_income from data["observations"]; convert date with pd.to_datetime and value with
 ↪  pd.to_numeric(errors="coerce"); show the last rows
fred_url = "https://api.stlouisfed.org/fred/series/observations"
params = {"series_id": "MHICO08013A052NCEN", "api_key": fred_key, "file_type": "json",
          "observation_end": "2026-09-30"}
data = requests.get(fred_url, params=params, headers=HEADERS).json()

df_income = pd.DataFrame(data["observations"])
df_income["date"] = pd.to_datetime(df_income["date"])
df_income["value"] = pd.to_numeric(df_income["value"], errors="coerce")
df_income.tail()
```

💬 **Discuss**

- This series is for Boulder County, `08013`; Step 2 asked about Boulder city, place `07850`. Why do the two incomes differ, and which would you report?
- FRED writes a missing value as `"."`. What would happen without `errors="coerce"`, and what would happen if you dropped it silently?
- FRED republishes the Census Bureau's SAIPE numbers. When would you go to the original source instead?

Step 6 — One function for FRED, and let the server do the math (12 minutes)

We'll ask FRED for five more series, and every request repeats Step 5's lines with a different series. So gather those lines into one function, `fred_series()`, written live: what changes from call to call becomes an argument (the series, the start date, `units`, and `frequency`), and the rest stays as Step 5 wrote it. A `frequency` of `None` means "as published": requests leaves out a parameter whose value is `None`.

Then use it. FRED can transform a series before it sends it (the chapter's "Server-Side Transformations"). With `units="pc1"`, the percent change from a year before, the Consumer Price Index (`CPIAUCSL`) comes back as the inflation rate. Plot it since 2020, with a line at the Federal Reserve's 2% target.

```
# Define fred_series(series_id, start, units="lin", frequency=None): Step 5's lines, gathered
#   params as in Step 5, plus observation_start=start, units=units, and frequency=frequency
#   Request it, build the DataFrame and convert as in Step 5, and return its date and value
 ↪  columns
def fred_series(series_id, start, units="lin", frequency=None):
    params = {"series_id": series_id, "api_key": fred_key, "file_type": "json",
              "observation_start": start, "observation_end": "2026-09-30",
              "units": units, "frequency": frequency}
    data = requests.get(fred_url, params=params, headers=HEADERS).json()
    df_series = pd.DataFrame(data["observations"])
    df_series["date"] = pd.to_datetime(df_series["date"])
    df_series["value"] = pd.to_numeric(df_series["value"], errors="coerce")
    return df_series[["date", "value"]]
```

```
# Ask fred_series() for CPIAUCSL since 2020-01-01, with units="pc1", into df_inflation
# On plt.subplots(), plot value against date, with a dashed red line at 2 labeled "2% target";
 ↪  title, axis labels, legend
df_inflation = fred_series("CPIAUCSL", "2020-01-01", units="pc1")

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(df_inflation["date"], df_inflation["value"], label="CPI, percent change")
ax.axhline(2, color="red", linestyle="--", label="2% target")
ax.set(title="Year-over-year inflation, United States (CPI)",
       xlabel="Month", ylabel="Percent change from a year before")
ax.legend()
plt.show()
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Check the server's math.

Do: find the month inflation peaked with `idxmax()`, ask for `CPIAUCSL`'s index levels (no `units`), and compute that month's percent change from a year before yourself. **Share:** the month, FRED's number, and yours.

```
# Microsprint: peak, the row of df_inflation with the largest value (idxmax)
# levels: fred_series() for CPIAUCSL since 2021-01-01 (no units), as a Series indexed by date
# Compute (level that month / level a year before - 1) * 100, and print it beside FRED's value
peak = df_inflation.loc[df_inflation["value"].idxmax()]
levels = fred_series("CPIAUCSL", "2021-01-01").set_index("date")["value"]
by_hand = (levels[peak["date"]] / levels[peak["date"] - pd.DateOffset(years=1)] - 1) * 100
print(f"{peak['date']:%B %Y}: FRED says {peak['value']:.2f}%, by hand {by_hand:.2f}%")
```

💬 Discuss

- Did your number match FRED's? What did `units="pc1"` save you, and what do you give up by letting the server do the math?
- Starting the chart in 2020 shapes what it shows. How would it look from 2015, or from 2022?
- In week 4 we peeled JSON one level at a time. Step 5's observations held more than a date and a value: what else, and why might FRED record it?

Step 7 — Denver against the nation (11 minutes)

National numbers hide local ones (the chapter's "Comparing CPI Across Locations"). The chapter asks for Denver's CPI as `CUURS49BSA0`, but FRED's CPI IDs reuse the Bureau of Labor Statistics' area codes, and in the BLS's list `S49B` is San Francisco–Oakland–Hayward; Denver–Aurora–Lakewood is `S48B`. So ask FRED's `series` endpoint, which describes a series rather than sending its data, what the chapter's ID is. The Denver-area CPI we'll use is `CUUSA433SA0`, which the Bureau of Labor Statistics publishes once a year. It isn't seasonally adjusted, so compare it, as the chapter says, with the national series that isn't either, `CPIAUCNS`, which FRED can average into years for you (`frequency="a"`). Ask for both as year-over-year changes since 2000, merge them on date, and plot them together. The merge keeps only the years both series have.

```
# Ask FRED's series endpoint, "https://api.stlouisfed.org/fred/series", about the chapter's ID,
↪  "CUURS49BSA0"
# Send series_id, api_key=fred_key, and file_type="json" with HEADERS; print the JSON: which city
↪  is it?
series_params = {"series_id": "CUURS49BSA0", "api_key": fred_key, "file_type": "json"}
print(requests.get("https://api.stlouisfed.org/fred/series", params=series_params,
↪  headers=HEADERS).json())
```

```
# df_denver: CUUSA433SA0; df_national: CPIAUCNS with frequency="a"; both since 2000-01-01, with
↪  units="pc1"
# df_cpi_compare: merge them on date, with suffixes ("_denver", "_national"); show its last rows
df_denver = fred_series("CUUSA433SA0", "2000-01-01", units="pc1")
df_national = fred_series("CPIAUCNS", "2000-01-01", units="pc1", frequency="a")
df_cpi_compare = df_denver.merge(df_national, on="date", suffixes=("_denver", "_national"))
df_cpi_compare.tail()
```

```
# On plt.subplots(), plot both lines and a dashed gray line at 2; title, axis labels, legend
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(df_cpi_compare["date"], df_cpi_compare["value_denver"], label="Denver area")
ax.plot(df_cpi_compare["date"], df_cpi_compare["value_national"], label="United States")
ax.axhline(2, color="gray", linestyle="--", label="2% target")
ax.set(title="Inflation, Denver area vs. United States",
       xlabel="Year", ylabel="Percent change from a year before")
ax.legend()
plt.show()
```

⏱️ Microsprint (5 minutes, alone or in pairs)

A short version of Additional Exercise 2, outside Colorado.

Do: search FRED for "consumer price index all items" and another metro area, and pick its annual series (the key uses the Seattle area, `CUUSA423SA0`). Merge it with `df_national`, and find the year it ran farthest above the nation. **Share:** the metro area, its series ID, the year, and the gap in percentage points.

```
# Microsprint: ask fred_series() for your metro area's annual CPI since 2000-01-01, with
↪  units="pc1"
# Merge it with df_national on date, and compute metro minus national for each year
# Print the year with the largest gap, and the gap
df_metro = fred_series("CUUSA423SA0", "2000-01-01", units="pc1")
gap = df_metro.merge(df_national, on="date", suffixes=("_metro", "_national"))
gap["above"] = gap["value_metro"] - gap["value_national"]
widest = gap.loc[gap["above"].idxmax()]
print(f"Seattle area, {widest['date'].year}: {widest['above']:.1f} points above the nation")
```

💬 Discuss

- Why compare a series that isn't seasonally adjusted with another that isn't? Denver's comes once a year, so we averaged the nation's into years: what did averaging hide?
- Which metro areas did the class find running hottest, and in which years? Is there a pattern?
- The chapter's Denver ID names another city, and its code would still have run and drawn a chart. What would have caught it, and what does that say about trusting code you copy, even from a textbook?

Step 8 — Put the two together (11 minutes)

The Census says what Boulder's housing is; FRED says what borrowing cost when it was built (the chapter's "Combining Census and FRED Data"). Ask the Census for Boulder's housing by decade back to the 1970s (`B25034_002E` to `B25034_007E`), and FRED for the 30-year mortgage rate as yearly averages (`MORTGAGE30US`, `frequency="a"`) since 1970. Plot them side by side.

One trap: the Census marks a value it can't publish with a code such as `-666666666`, not a blank. The chapter's `census_int()` turns such codes into `None` one value at a time; pandas's `mask()` does it for a whole Series in one line, since no count is below zero.

```
# housing_params: NAME and B25034_002E through B25034_007E for Boulder (place:07850 in state:08);
 ↪  request it into housing_data
# counts: pd.to_numeric of housing_data[1][1:7], as a Series indexed by decade: 2020, 2010, 2000,
 ↪  1990, 1980, 1970
# Mask every count below zero, and show counts
housing_params = {
    "get": "NAME,B25034_002E,B25034_003E,B25034_004E,B25034_005E,B25034_006E,B25034_007E",
    "for": "place:07850", "in": "state:08", "key": census_key}
housing_data = requests.get(census_url, params=housing_params, headers=HEADERS).json()

counts = pd.Series(pd.to_numeric(housing_data[1][1:7]), index=[2020, 2010, 2000, 1990, 1980,
 ↪  1970])
counts = counts.mask(counts < 0)
counts
```

```
2020      68
2010    3784
2000    3847
1990    5931
1980    7400
1970    8980
dtype: int64
```

```
# df_mortgage: fred_series() for MORTGAGE30US since 1970-01-01, with frequency="a"
# Two panels from plt.subplots(1, 2): bars of counts by decade, and the mortgage rate over time;
 ↪  titles and axis labels on both
df_mortgage = fred_series("MORTGAGE30US", "1970-01-01", frequency="a")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.5))
ax1.bar(counts.index, counts, width=8, color="steelblue")
ax1.set(title="Boulder housing by decade built",
        xlabel="Decade built (its first year)", ylabel="Housing units")
ax2.plot(df_mortgage["date"], df_mortgage["value"], color="darkred")
ax2.set(title="30-year mortgage rate (annual average)",
        xlabel="Year", ylabel="Interest rate (%)")
plt.show()
```

⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 4: put the two sources in one table.

Do: average the mortgage rate within each decade from the 1970s to the 2010s, put it beside the units Boulder built that decade, and compute the correlation. **Share:** the correlation, its sign, and how many data points it rests on.

```
# Microsprint: add a decade column to df_mortgage (the year // 10 * 10), and average value by decade
# df_decades: those averages beside counts in one DataFrame, from 1970 to 2010
# Print the correlation between its two columns and the number of decades; show df_decades
df_mortgage["decade"] = df_mortgage["date"].dt.year // 10 * 10
rate_by_decade = df_mortgage.groupby("decade")["value"].mean()
df_decades = pd.DataFrame({"mortgage_rate": rate_by_decade, "units_built": counts}).loc[1970:2010]
r = df_decades["mortgage_rate"].corr(df_decades["units_built"])
print(f"Correlation: {r:.2f}, from {len(df_decades)} decades")
df_decades.round(2)
```

💬 Discuss

- The chapter says low-rate decades built more. Does Boulder's table agree, and what can a correlation from five points tell you?
- The Census counts only the homes still standing in 2022. How does that bias a comparison of the 1970s with the 2010s?
- In week 9 we checked PDF extractions with counts and sums. What check would have caught a `-666666666` before it reached a chart?

Step 9 — Optional: campaign finance from the FEC (if there's time)

The Federal Election Commission's API needs a third key, `FEC_API_KEY` (the chapter's "FEC API"). Search for candidates by name, then ask for one candidate's totals. Candidate IDs start with the office sought (`P` for president, `S` for Senate, `H` for House); committee IDs start with `C`. Contribution records name individual donors: count them, don't print them.

```
# Read FEC_API_KEY into fec_key with os.environ[...]
# Request https://api.open.fec.gov/v1/candidates/search/ with q="Biden", office="P", election_year=2024, per_page=5, api_key=fec_key
# Print each result's name, party_full, and candidate_id
fec_key = os.environ["FEC_API_KEY"]
fec_search_url = "https://api.open.fec.gov/v1/candidates/search/"
params = {"q": "Biden", "office": "P", "election_year": 2024, "per_page": 5, "api_key": fec_key}
data = requests.get(fec_search_url, params=params, headers=HEADERS).json()
for candidate in data["results"]:
    print(f"{candidate['name']} ({candidate['party_full']}), ID {candidate['candidate_id']}")
```

```
# Request https://api.open.fec.gov/v1/candidate/P80000722/totals/ with api_key=fec_key and election_year=2024
# Print each result's cycle, receipts, and disbursements, with dollar signs and commas
fec_totals_url = "https://api.open.fec.gov/v1/candidate/P80000722/totals/"
params = {"api_key": fec_key, "election_year": 2024}
data = requests.get(fec_totals_url, params=params, headers=HEADERS).json()
for result in data["results"]:
    print(f"Cycle {result['cycle']}: receipts ${result['receipts']:,.0f}, "
          f"disbursements ${result['disbursements']:,.0f}")
```

💬 Discuss

- Campaign finance records name donors. How does that change what you can publish, compared with counts of homes?
- The FEC's data exists because a 1971 law requires it. In week 3's terms, which value does that serve, and what pressure could erode it?

Step 10 — What did we learn? (5 minutes)

In a few sentences, as comments:

- Which decade built the most of Boulder's housing, and which the least? Does the mortgage rate explain the pattern, or does something else, such as the city's limits on growth?
- When did Denver's inflation run above the nation's, and by how much at most?
- Where did each API make you look something up: a variable code, a series ID, a FIPS code? Which lookup went wrong, and how did you find out?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, load your keys and confirm them without printing them: Step 0 read them from the environment, and a missing one stops the cell.
- Step 2, query every Colorado county: Steps 2 and 3 built the `get`, `for`, and `in` parameters, and Step 1 the catalog for choosing a variable.
- Step 3, build and clean the DataFrame: Step 2's header row and `pd.to_numeric`, and Step 8's `mask()` for codes like `-666666666`.
- Step 4, a bar chart of the top 20 counties: Step 3's labeled bar chart.
- Step 5, a related FRED series for Colorado: Step 5's request and Step 6's `fred_series()`, and Step 7's microsprint, which searched FRED for a series ID.
- Step 6, interpret: Step 4's sanity check and margins of error, and Step 8's question of what two sources side by side can and can't show.

```
# Your answers here, as comments
# Of the decades Step 8 asked for, the 1970s built the most of the Boulder housing still standing
#   (8,980 units), then the 1980s (7,400) and the 1990s (5,931); the 2010s built the least of the
 ↪  full
#   decades (3,784, just under the 2000s' 3,847), and "2020 or later" has 68, for 2020-2022 only.
 ↪  So the
#   chapter's "significant construction in the 1970s and 1990s" passes over the 1980s. Growth
 ↪  limits fit
#   Step 3's microsprint: 16.5% of Boulder's housing is from 2000 or later, against 30.1% of
 ↪  Longmont's and
#   68.5% of Castle Rock's, though the lab doesn't test the cause. Whether the mortgage rate
 ↪  explains the
#   pattern comes from Step 8's FRED cells and microsprint: to write once they have run.
# Denver against the nation: to write from Step 7 once FRED's cells have run.
# Lookups: Step 1's catalog for variable codes (B25034, B19013), FIPS codes for places (Boulder
#   07850, Longmont 45970, in state 08), and FRED's search for series IDs. The one that went
 ↪  wrong is
#   the chapter's Denver series, CUURS49BSA0: in the Bureau of Labor Statistics' area codes, S49B
 ↪  is
#   San Francisco-Oakland-Hayward, and Denver is S48B. Step 7's first cell asks FRED what the ID
 ↪  is; it
#   hasn't run yet. And a number can be right and still mislead: Step 4's microsprint found
 ↪  Boulder's
#   median income ($80,243, +/- $3,561) below Longmont's ($89,720, +/- $3,791), with intervals
 ↪  that
#   don't overlap (Boulder's tops out at $83,804, Longmont's starts at $85,929).
```

Friday: from the lab to the textbook (15 minutes)

Chapter 11 is the textbook's, and you can improve it. This lab turned up several candidates; pick one, or something you found yourself, and propose a revision as `handouts/common/revision-framework.md` describes.

- **A series ID for the wrong city.** The chapter's Denver series, `CUURS49BSA0`, is San Francisco's in the Bureau of Labor Statistics' area codes (Step 7). What should the chapter use instead, and what should it say about the series' frequency?
- **A paragraph the data doesn't support.** After its two-panel chart, the chapter says Boulder saw "significant construction in the 1970s and 1990s," and that low-rate decades built more. Compare that with Step 8's chart and its microsprint.
- **A 200 that isn't data.** The Census Bureau redirects a bad key to a web page, and FRED answers during maintenance with one; both arrive with status 200. The chapter's code, like ours, calls `.json()` straight away, and its "Common Issues to Debug" doesn't mention this. What would a reader see, and what should the chapter say?
- **A microsprint that would make a good exercise.** The margin-of-error check (Step 4) or the hand calculation of `pc1` (Step 6).